📓 Programação para Internet II

📅 Aula 03 — JavaScript: Variáveis e Tipos de Dados

🎯 Objetivo da Aula

- Compreender o conceito de variáveis em JavaScript.
- Aprender a declarar variáveis usando `var`, `let` e `const`.
- Entender a importância das convenções de nomenclatura de variáveis.
- Conhecer os diferentes tipos de dados em JavaScript (primitivos e não-primitivos).
- Utilizar o operador `typeof` para verificar o tipo de uma variável.
- Praticar a saída de dados com `console.log()`.

🧠 Revisão: O Papel do JavaScript na Web

Como vimos na Aula 01, o JavaScript é a linguagem que traz **interatividade e dinamismo** para nossas páginas web. Ele permite que a página "reaja" às ações do usuário, manipule o conteúdo HTML e o estilo CSS, e muito mais.

Para que o JavaScript possa fazer tudo isso, ele precisa de uma forma de **armazenar informações** e **manipular dados**. É aí que entram as **variáveis** e os **tipos de dados**.

📝 Variáveis: Armazenando Informações

> **Variáveis** são como "caixas" nomeadas na memória do computador onde podemos guardar valores. Elas nos permitem referenciar esses valores por um nome, facilitando a leitura e a manipulação dos dados em nosso código.

Pense em uma variável como um rótulo que você cola em uma caixa. Dentro da caixa, você pode colocar qualquer coisa (um número, um texto, etc.). O rótulo é o nome da variável, e o que está dentro da caixa é o seu valor.

🔑 Declaração de Variáveis

Em JavaScript, podemos declarar variáveis de três formas principais: `var`, `let` e `const`. Cada uma possui características específicas que afetam como e onde a variável pode ser usada (escopo e mutabilidade).

`var` (Antigo e Menos Recomendado)

- Foi a forma original de declarar variáveis.
- Possui **escopo de função** ou **global**. Isso significa que uma variável declarada com `var` dentro de um bloco de código (como um `if` ou `for`) ainda será visível fora dele, o que pode levar a erros

inesperados.
- Pode ser **redeclarada** e **reatribuída**.

```
// Declaração e atribuição
var nome = "João";
console.log(nome); // Saída: João

// Reatribuição
nome = "Maria";
console.log(nome); // Saída: Maria

// Redeclaração (válida, mas pode causar confusão)
var nome = "Pedro";
console.log(nome); // Saída: Pedro

// Exemplo de escopo (problema do var)
if (true) {
  var saudacao = "Olá!";
}
console.log(saudacao); // Saída: Olá! (Funciona fora do bloco!)
```

`let` (Recomendado para Variáveis Mutáveis)

- Introduzido no ES6 (ECMAScript 2015) para resolver problemas do `var`.
- Possui **escopo de bloco**. Uma variável declarada com `let` dentro de um bloco (`{ }`) só é acessível dentro daquele bloco.
- Não pode ser **redeclarada** no mesmo escopo, mas pode ser **reatribuída**.

```
let idade = 30;
console.log(idade); // Saída: 30

idade = 31; // Reatribuição é permitida
console.log(idade); // Saída: 31

// let idade = 32; // ERRO: 'idade' já foi declarada

// Exemplo de escopo de bloco com let
if (true) {
  let mensagem = "Tudo bem?";
  console.log(mensagem); // Saída: Tudo bem?
}
// console.log(mensagem); // ERRO: mensagem is not defined (Não funciona fora do
bloco!)
```

`const` (Recomendado para Variáveis Imutáveis/Constantes)

- Também introduzido no ES6.
- Possui **escopo de bloco**.

- Não pode ser **redeclarada** nem **reatribuída** após sua inicialização. Deve ser inicializada no momento da declaração.
- O nome `const` vem de "constante", indicando que seu valor não deve mudar.

```
const PI = 3.14159;
console.log(PI); // Saída: 3.14159

// PI = 3.14; // ERRO: Assignment to constant variable. (Não pode ser reatribuída!)

const NOME_COMPLETO = "Ana Silva";
console.log(NOME_COMPLETO); // Saída: Ana Silva

// const semValor; // ERRO: Missing initializer in const declaration. (Deve ser inicializada!)
```

Importante: Embora o valor de uma `const` não possa ser reatribuído, se a `const` for um objeto ou um array, o *conteúdo* desse objeto ou array ainda pode ser modificado. O que a `const` garante é que a *referência* àquele objeto/array não mudará.

🏷️ Convenções de Nomenclatura

- **Camel Case:** É a convenção mais comum em JavaScript. A primeira palavra começa com letra minúscula e as seguintes com letra maiúscula (ex: `nomeCompleto`, `idadeDoUsuario`).
- Não pode começar com números.
- Não pode conter espaços ou caracteres especiais (exceto `_` e `$`).
- Nomes devem ser descritivos e significativos.

```
let primeiroNome = "Carlos";
let totalVendas = 1500.75;
const GRAVIDADE = 9.81; // Constantes são frequentemente em UPPER_SNAKE_CASE
```

📊 Tipos de Dados em JavaScript

JavaScript é uma linguagem de tipagem dinâmica, o que significa que você não precisa especificar o tipo de dado ao declarar uma variável (o interpretador do JS descobre o tipo em tempo de execução).

Existem 7 tipos de dados primitivos e 1 tipo de dado não-primitivo (Objeto).

🧩 Tipos de Dados Primitivos (Atomic Values)

Os tipos primitivos são dados que representam um único valor.

1. `Number` (Números)

- Representa números inteiros e de ponto flutuante (decimais).
- Não há distinção entre int e float como em outras linguagens.

```
let idade = 25;        // Número inteiro
let preco = 19.99;     // Número de ponto flutuante
let temperatura = -5; // Números negativos
```

2. `String` (Textos)

- Representa sequências de caracteres (textos).
- Podem ser delimitadas por aspas simples (`' '`), aspas duplas (`" "`) ou template literals (crases `` ` ` ``).

```
let nome = "Alice";
let sobrenome = 'Silva';
let frase = `Olá, ${nome} ${sobrenome}!`; // Template literal permite interpolação
console.log(frase); // Saída: Olá, Alice Silva!
```

3. `Boolean` (Verdadeiro/Falso)

- Representa valores lógicos: `true` (verdadeiro) ou `false` (falso).
- Usado para condições e lógica.

```
let estaLogado = true;
let isMaiorDeIdade = false;
```

4. `Undefined` (Indefinido)

- Representa uma variável que foi declarada, mas ainda não teve um valor atribuído.
- É o valor padrão de variáveis recém-declaradas.

```
let valor;
console.log(valor); // Saída: undefined
```

5. `Null` (Nulo)

- Representa a ausência intencional de qualquer valor de objeto.
- É um valor que você *atribui* para indicar "nada" ou "vazio".

```
let usuarioLogado = null; // Indica que não há usuário logado
```

Diferença entre `undefined` e `null`: `undefined` significa que uma variável não tem um valor (não foi inicializada), enquanto `null` significa que a variável *tem* um valor, e esse valor é explicitamente "nada".

6. `Symbol` (Símbolo - ES6)

- Representa um identificador único e imutável.
- Usado principalmente em situações onde a unicidade é importante, como chaves de propriedades de objeto para evitar colisões.
- O principal uso é criar chaves únicas, especialmente em objetos, evitando conflitos.

```
const id = Symbol('id');
const id2 = Symbol('id');
console.log(id === id2); // Saída: false (são únicos)
```

7. `BigInt` (Números Inteiros Grandes - ES2020)

- Permite representar números inteiros com precisão arbitrária (maiores que o limite do `Number`).
- Declarado adicionando `n` ao final do número.

```
const numeroMuitoGrande = 9007199254740991n;
console.log(numeroMuitoGrande);
```

📦 Tipo de Dado Não-Primitivo: `Object` (Objeto)

- Representa coleções de dados mais complexos e entidades mais elaboradas.
- Inclui objetos literais (`{}`), arrays (`[]`), funções (`function`), entre outros.
- Ao contrário dos primitivos, objetos são armazenados por referência.

```
// Objeto Literal
let pessoa = {
  nome: "Carlos",
  idade: 30,
  profissao: "Programador"
};
console.log(pessoa.nome); // Saída: Carlos

// Array (um tipo especial de objeto)
let frutas = ["Maçã", "Banana", "Laranja"];
console.log(frutas[0]); // Saída: Maçã

// Função (também um tipo especial de objeto)
function saudacao(nome) {
  return `Olá, ${nome}!`;
}
console.log(saudacao("Fernanda")); // Saída: Olá, Fernanda!
```

🕵 Verificando Tipos de Dados com `typeof`

O operador `typeof` retorna uma string indicando o tipo do operando. É útil para depuração e para tomar decisões baseadas no tipo de dado.

```
console.log(typeof 10);              // Saída: "number"
console.log(typeof 3.14);            // Saída: "number"
console.log(typeof "Olá");           // Saída: "string"
console.log(typeof true);            // Saída: "boolean"
console.log(typeof undefined);       // Saída: "undefined"
console.log(typeof Symbol('abc'));   // Saída: "symbol"
console.log(typeof 10n);             // Saída: "bigint"

console.log(typeof null);            // Saída: "object" (OBS: Isso é um bug
histórico do JS!)
console.log(typeof { a: 1 });        // Saída: "object"
console.log(typeof [1, 2, 3]);       // Saída: "object"
console.log(typeof function() {});   // Saída: "function" (Funções são objetos
callable)
```

Observação importante sobre `typeof null`: Historicamente, `typeof null` retorna `"object"`. Este é um erro conhecido na linguagem que foi mantido por compatibilidade. Para verificar se um valor é `null`, você deve usar `=== null`.

💻 Saída de Dados com `console.log()`

Durante o desenvolvimento, `console.log()` é uma ferramenta essencial para exibir valores de variáveis, o resultado de expressões e mensagens de depuração no console do navegador ou do Node.js.

```
let nome = "professor";
let idade = 40;
let estaAtivo = true;

console.log("Variável nome:", nome);
console.log("A idade é:", idade);
console.log("Está ativo?", estaAtivo);
console.log("O tipo de idade é:", typeof idade);

// Concatenando e usando template literals
console.log("Meu nome é " + nome + " e tenho " + idade + " anos.");
console.log(`Meu nome é ${nome} e tenho ${idade} anos.`);
```

🧠 O que aprendemos hoje?

- Variáveis são espaços na memória para armazenar dados.
- Usamos `let` para variáveis que podem mudar de valor e `const` para valores constantes. `var` é obsoleto.
- JavaScript tem 7 tipos primitivos (`Number`, `String`, `Boolean`, `Undefined`, `Null`, `Symbol`, `BigInt`) e o tipo `Object` para dados mais complexos.
- O operador `typeof` nos ajuda a identificar o tipo de dado de uma variável.
- `console.log()` é fundamental para depurar e entender o fluxo do programa.

🧩 Desafios Práticos

1. **Declaração e Atribuição:**
    - Declare uma variável `nomeAluno` com seu nome usando `let`.
    - Declare uma variável `nota1` com um número decimal (ex: 8.5) usando `let`.
    - Declare uma constante `MEDIA_APROVACAO` com o valor 7 usando `const`.
    - Imprima no console o valor e o tipo de cada uma dessas variáveis.
2. **Modificação de Variáveis:**
    - Altere o valor de `nomeAluno` para "Fulano de Tal".
    - Altere o valor de `nota1` para 9.0.
    - Tente alterar o valor de `MEDIA_APROVACAO` e observe o erro no console.
3. **Explorando Tipos:**
    - Crie uma variável `estaChovendo` e atribua um valor booleano a ela.
    - Crie uma variável `minhaAltura` e atribua sua altura em metros.
    - Crie uma variável `meuObjeto` e atribua um objeto literal vazio `{}` a ela.
    - Use `console.log()` e `typeof` para exibir o valor e o tipo de cada uma.
4. **Template Literals:**
    - Usando as variáveis `nomeAluno` e `nota1`, crie uma frase que diga "O aluno [nomeAluno] tirou [nota1] na prova." utilizando template literals e imprima no console.

📌 Próxima Aula

> Na próxima aula, vamos aprofundar no mundo dos **Operadores e Expressões** em JavaScript, aprendendo como realizar cálculos, comparações e operações lógicas com os dados que armazenamos em nossas variáveis.